TEXAS TECH UNIVERSITY
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
CAPSTONE PROJECT LABORATORY

# Infrastructure-Free Vehicle-to-Vehicle Tracking: Design and Analysis of a LoRa Position Link Between Two Heltec WiFi LoRa 32 V4 Nodes with Dedicated Dashboard Displays

*A Technical Report on the Off-Grid Vehicle Tracker*

Group 9

Deo · Josiah · Victor · Cannon

Sponsor: Dr. Richard Gale

Fall 2026

## Abstract

This report explains how two vehicles track each other with no cellular network, no internet, and no infrastructure of any kind between them. Each vehicle carries a Heltec WiFi LoRa 32 V4 radio node running the open-source Meshtastic firmware and a Waveshare ESP32-S3 reflective-LCD dashboard display running firmware written by this team. The radios learn their own positions from GNSS satellites and broadcast them to each other over a 915 MHz LoRa link; each display receives the decoded traffic from its local radio over Bluetooth Low Energy and converts it into a live navigation screen showing the partner's direction, distance, and speed. The report develops the complete chain that makes this possible: the chirp spread spectrum physical layer that decodes signals 17.5 dB below the noise floor, the time-on-air arithmetic that prices every packet at roughly 0.56 seconds, the 161 dB link budget that supports a 3 to 5 km design range, the Meshtastic channel contract in which a hashed channel name becomes the operating frequency, and the published Bluetooth Client API that allowed a custom display to attach to unmodified radio firmware. All claims are tied to measurements made on this hardware, and all mathematics is worked with this system's actual parameters.



## Acknowledgement

This project is carried out collaboratively by the four members of Group 9, each contributing complementary expertise. Deo Mwala serves as software lead for the protocol layer: the Bluetooth Low Energy client that connects each display to its radio, the protocol-buffer decoding of the Meshtastic stream, and the node table that turns packets into tracked state. Josiah leads the display and navigation firmware, including the navigation mathematics of Section 8 and the three-page driver interface. Victor is the radio and hardware lead, responsible for provisioning and configuring the Meshtastic radios, antenna selection, and the range testing program. Cannon is the systems and test lead, responsible for integration testing and the verification record of Section 10. The team thanks its sponsor, Dr. Richard Gale, for his guidance and for the weekly design reviews that shaped this system, and acknowledges the Meshtastic open-source community, whose documented firmware and published Client API form the foundation this project builds on.


## 1. Introduction

Modern vehicle tracking assumes infrastructure. A phone reports its position through a cell tower; a fleet tracker reports through a satellite service; both stop working exactly where tracking matters most, on desert trails, in mountain valleys, and on rural highways beyond coverage. The goal of this project is a system in which two vehicles always know each other's position using nothing but the equipment they carry. There are no accounts, no subscriptions, and no third party between them.

Five ideas organize this entire report, and essentially every behavior and every failure of the system traces back to one of them.

1. **Agreement is the connection.** There is no session, pairing, or handshake between the radios; they exchange data exactly when they independently agree on frequency, modulation, and encryption

key. This is the report's thesis, and every failure in Section 9 is one of these agreements quietly broken.

2. **The name is the frequency.** The radios store no frequency; they derive one by hashing the channel name, so checking one number on a screen verifies the deepest layer of the agreement (Section 6.1).
3. **Decibels are ratios, and LoRa decodes below the noise.** Signal quantities are logarithmic, every 6 dB of margin roughly doubles range, and the modulation reads signals quieter than the noise floor itself (Sections 4 and 5).
4. **Online is arithmetic, not a connection.** Every liveness indicator is a comparison between the current time and a stored timestamp, so when an indicator lies, a timestamp is lying (Section 6.4).
5. **Airtime is the budget.** Only one node can speak at a time and every packet costs real channel time, so reporting cadence is a resource allocation problem, not a preference (Section 6.3).

Every design decision in the following sections either establishes one of those agreements, budgets the physical resources that keep the agreement usable, or displays the agreed information honestly to a driver. Section 2 defines the architecture. Sections 3 through 5 build the radio node and its physical layer from the datasheet upward, in the same order the signal itself is built. Section 6 explains the protocol layer that turns raw packets into a private, self-managing channel. Sections 7 and 8 cross into the display node and the navigation mathematics it computes. Section 9 presents the failures encountered on real hardware and the rules they taught, Section 10 reports the experimental verification, Section 11 outlines future work, and Section 12 concludes.

The work described here was divided among the four members of Group 9, and each technical section below carries the name of the member responsible for that part of the system. Victor, the radio and hardware lead, configured the radios and characterized the physical layer (Sections 3 and 4). Cannon, the systems and test lead, built the measurement harness behind the link budget and led the drive testing (Sections 5 and 10, the latter jointly with Victor). Deo wrote the protocol firmware, including the Bluetooth client and packet decoding that connect the two nodes (Section 6 and Section 7.2). Josiah wrote the display and navigation firmware that turns decoded packets into the driver's screen (Sections 7 and 8). The remaining sections, including the architecture, the failure catalog, and this introduction, are joint work.

## 2. System Architecture

The system consists of four embedded devices arranged in two identical halves, one per vehicle. Each half contains a radio node and a display node.

| Device | Hardware | Firmware | Role |
|---|---|---|---|
| Radio node (2) | Heltec WiFi LoRa 32 V4 | Stock Meshtastic 2.7.26 | Acquires its own GNSS fix; broadcasts position, telemetry, and text over LoRa; serves decoded traffic to its display over BLE |
| Display node (2) | Waveshare ESP32-S3-RLCD-4.2 | Custom, written by Group 9 | Connects to its local radio over BLE; decodes the protocol stream; renders direction, distance, speed, messages, and warnings |

Figure 1, presented on its own page at the end of this section, shows the complete signal path. Information flows in one direction around a loop. Vehicle A's radio measures its position and broadcasts it over LoRa; vehicle B's radio receives and decrypts the packet and holds it in a node database; vehicle B's display drains that database over Bluetooth Low Energy and renders the result. The same path runs simultaneously in the opposite direction. The deliberate consequence of this architecture is that the two engineering problems are separable: the radio link is a solved, heavily tested open-source system that this team configures and characterizes but does not modify, while the display is entirely this team's code attached at a published interface. Section 7.2 examines that interface, because it is the single decision that made the project feasible in one semester.

### 2.1 Design requirements

The requirements below were fixed early with the sponsor and drive every quantitative choice in this report. Each one traces to the section that satisfies it, and the verification program of Section 10 tests against them directly.

| Requirement | Target | Addressed in |
|---|---|---|
| Operate with no infrastructure | No cellular, internet, or third-party service in the data path | Entire architecture (Section 2) |
| Useful range in real terrain | 3 to 5 km with buildings and terrain in the way | Link budget (Section 5) |
| Fresh positions while driving | Updates at least every 10 seconds and 20 meters when moving | Cadence contract (Section 6.3) |
| Readable on a dashboard | Legible in direct sunlight without glare at night | Reflective display (Section 7.1) |
| Two-way messaging | Preset texts deliverable without typing while driving | Protocol layer and display (Sections 6 and 7) |
| Honest staleness | The driver can always tell measurement from estimate | Liveness semantics and dead reckoning (Sections 6.4 and 8.3) |
| Privacy from casual interception | No position or message readable by a third party | Encryption model (Section 6.2) |
| Independent power | A full day of operation on internal batteries | Node hardware (Sections 3 and 7.1) |

## Figure 1. The System Architecture

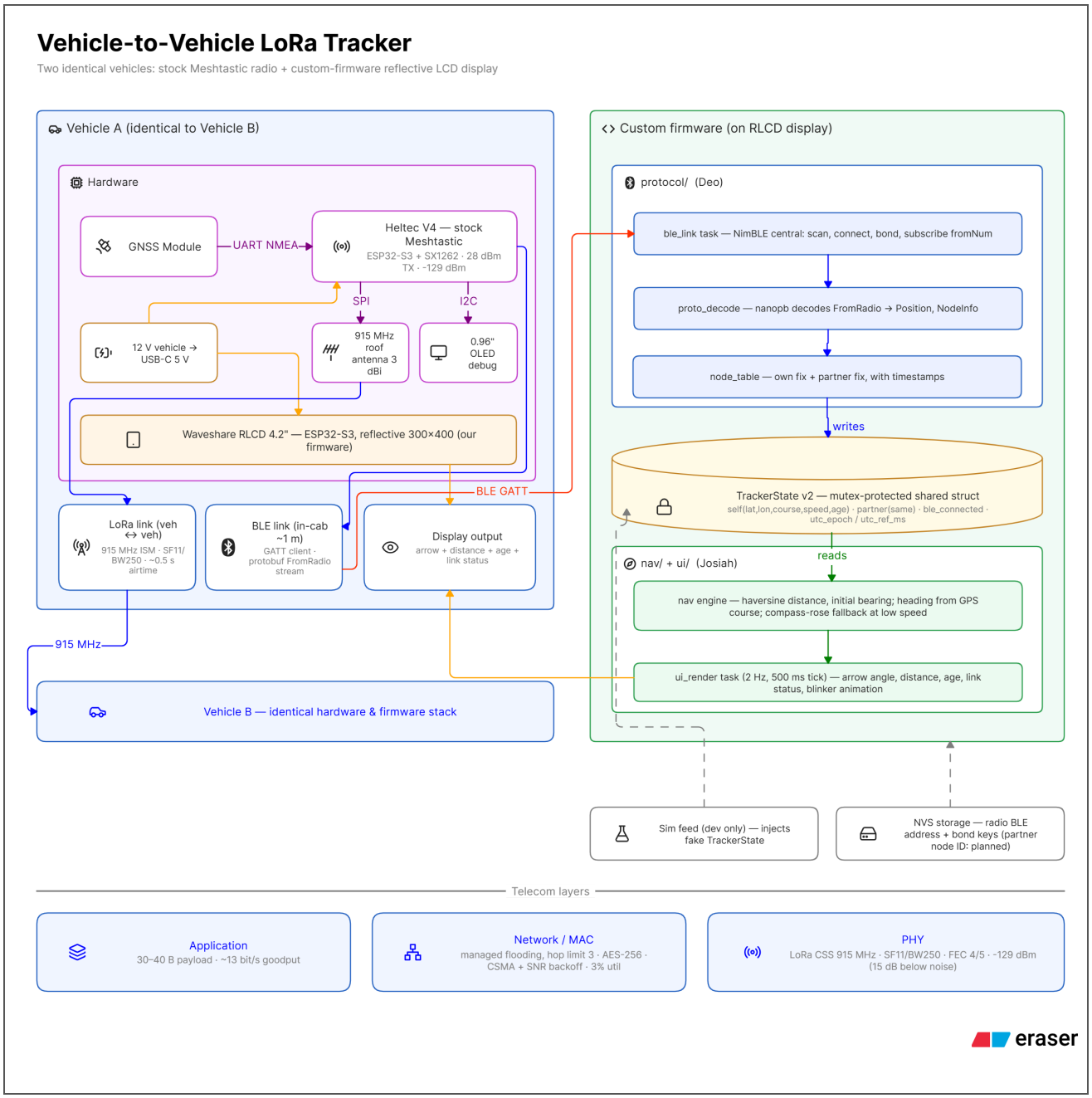



Figure 1. Complete system architecture of the Off-Grid Vehicle Tracker. Each vehicle pairs a stock Meshtastic radio with a custom-firmware reflective display; the vehicles share only the 915 MHz LoRa channel.

The diagram reads left to right, following the data. The left half is vehicle A's hardware: the GNSS module feeds the Heltec radio over a serial NMEA link, the radio drives its LoRa transceiver over SPI and its status OLED over I2C, and vehicle power reaches both devices through a 12 V to USB-C adapter. The radio owns both wireless links that leave the box: the 915 MHz LoRa channel to the partner vehicle at the bottom left, and the short-range Bluetooth GATT link that carries the decoded packet stream to the Waveshare display.

The right half is the firmware this team wrote, drawn as the two tasks it actually runs. The protocol side connects, decodes each FromRadio protobuf, and maintains the node table; it writes its results into TrackerState, the mutex-protected structure in the center that is the single interface between the halves. The navigation and interface side only reads: it computes distance and bearing from the two stored fixes and renders the three screen pages at two frames per second. One writer, one reader, one shared structure; that discipline is why the firmware has no data races by construction.

The strip along the bottom places the system in telecom terms: an application layer of 30 to 40 byte position payloads, a network layer of managed flooding with CSMA and a hop limit, and a LoRa chirp spread spectrum physical layer decoding 17.5 dB below the noise floor. Vehicle B, noted at the lower left, runs an identical hardware and firmware stack, which is what makes the whole system symmetric: every arrow in this figure exists twice, once in each direction.

## 3. The Radio Node: Heltec WiFi LoRa 32 V4 *(Victor)*

The Heltec V4 is a development board that integrates the three subsystems a tracking node needs. An Espressif ESP32-S3 microcontroller, a dual-core Xtensa LX7 at 240 MHz with 8 MB of flash, runs the Meshtastic firmware and its FreeRTOS task set (Espressif). A Semtech SX1262 transceiver provides the sub-GHz LoRa physical layer; it is a half-duplex radio covering 150 to 960 MHz, programmable in spreading factor, bandwidth, and coding rate, with a receive sensitivity that reaches -131 dBm at the settings used here (Semtech, *SX1261/2*). A u-blox GNSS module supplies position fixes, typically converging to a five-satellite fix within a minute under open sky. The board carries a 0.96-inch OLED status screen, a USB-C port for provisioning, and a lithium cell for operation away from vehicle power.

### 3.1 Inside the transceiver

Figure 2 reproduces the SX1262's internal block diagram from the datasheet, and it rewards a careful walk from left to right because the whole receive path of Section 4 lives inside it. The antenna enters through a matching network and low-pass filter. On receive, a low-noise amplifier lifts the faint arriving signal, a mixer driven by the phase-locked loop translates it down from 913 MHz, and an analog-to-digital converter hands the result to one of two demodulators, the LoRa modem used here or a conventional FSK modem. The protocol engine assembles demodulated bits into packets in the data buffer, which the ESP32-S3 reads over a four-wire SPI bus. On transmit the path runs in reverse, ending in the power amplifier. The crystal oscillator and PLL that set the carrier frequency are shared by both directions, which matters in Section 6: when Meshtastic computes a frequency from the channel name, this PLL is the hardware that must be programmed to it, and the OLED frequency readout this team treats as ground truth is a report of that programming. The DC-DC converter and low-dropout regulator in the corner of the diagram power the chip, and their efficiency is part of why a node runs for days on a small cell.

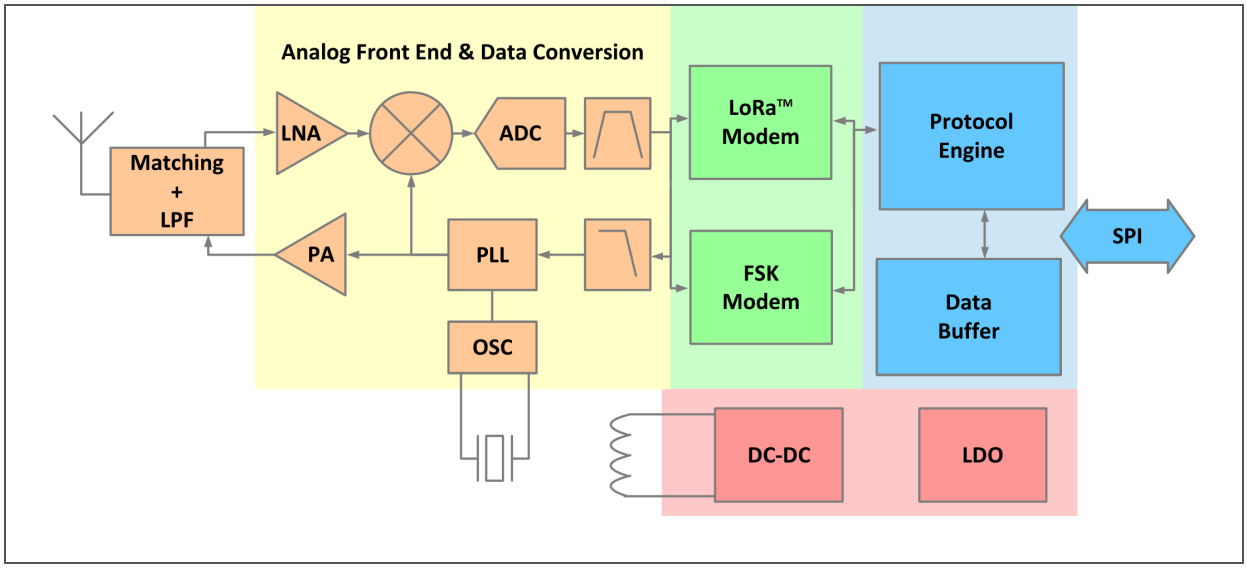



Figure 2. Internal block diagram of the SX1261/2 transceiver, adapted from the datasheet, revision 2.2, figure 4-1 (Semtech, *SX1261/2*).

Three facts from this datasheet shape the whole design. First, sensitivity is not a single number but a function of the modulation parameters; the -131 dBm figure used in Section 5 belongs specifically to the SF11, 250 kHz operating point, so choosing those parameters is choosing that sensitivity. Second, the bare SX1262 tops out at +22 dBm, and the Heltec board adds amplification to reach the +30 dBm the configuration commands, which happens to be the legal ceiling for this band under 47 C.F.R. § 15.247 (United States). Third, the radio is half duplex, and the datasheet's own operating-mode diagram, reproduced in Figure 3, shows why.

### 3.2 The transceiver state machine

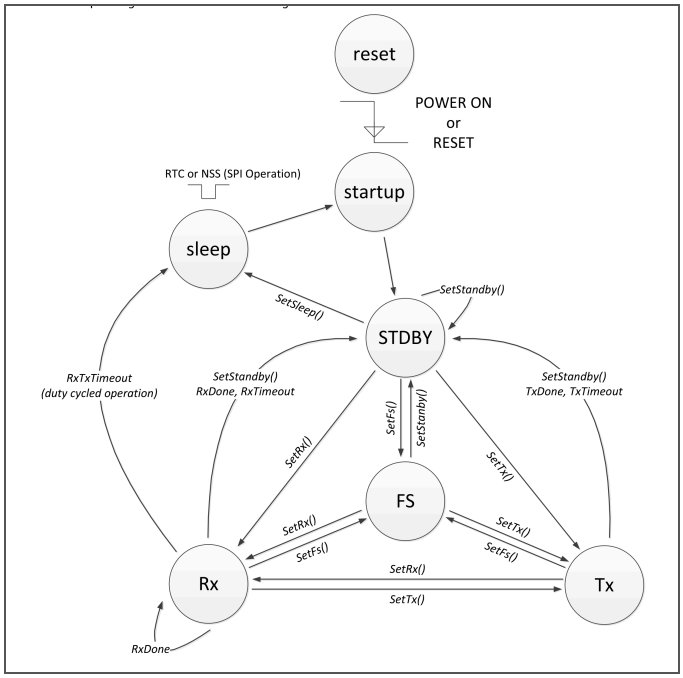



Figure 3. Transceiver circuit modes of the SX1261/2, adapted from the datasheet, revision 2.2, figure 9-1 (Semtech, *SX1261/2*).

The chip is always in exactly one of the circuit modes in Figure 3. From power-on it passes through startup into standby, where the host configures it; the frequency-synthesis mode locks the PLL onto the carrier; and the Rx and Tx states are where the radio actually listens and speaks. The diagram contains no state in which the chip does both, and the Rx and Tx circles are connected only through explicit commands. That single picture is the physical root of a constraint the protocol layer inherits: while a node spends its 0.56 seconds in Tx transmitting a position packet, it is deaf, so two nodes transmitting at once on one channel simply lose each other's packets. Meshtastic manages this with listen-before-talk and randomized backoff, but no protocol can repeal the state machine, which is why Section 6.3 treats airtime as a budget to be rationed rather than a convenience. The sleep state matters too; Meshtastic parks the radio there between duties, which is where the days-long battery figures come from.

## 4. The Physical Layer: LoRa Chirp Spread Spectrum *(Victor)*

### 4.1 How a chirp carries bits

LoRa (Long Range) is a chirp spread spectrum modulation. Each symbol is a frequency sweep that crosses the entire channel bandwidth, and the information is carried in the sweep's cyclic starting offset. With spreading factor SF, the sweep is divided into $2^{SF}$ distinguishable offsets, so one symbol carries SF bits. This system uses the Meshtastic LONG_FAST preset: SF = 11, bandwidth BW = 250 kHz, and coding rate 4:5 (Meshtastic Project, "Radio Settings"). A spreading factor of 11 gives $2^{11}$ = 2048 offsets and therefore 11 bits per symbol. The duration of one symbol is the chip count divided by the bandwidth:

$$T_{sym} = \frac{2^{SF}}{BW} = \frac{2^{11}}{250{,}000\ \text{Hz}} = 8.19\ \text{ms}$$

The receiver demodulates by multiplying the incoming signal with a locally generated inverse chirp, which collapses each sweep into a single frequency bin recoverable by a Fourier transform. Because this de-chirping integrates all 2048 chips coherently, the processing gain allows the SX1262 to demodulate at a signal-to-noise ratio of -17.5 dB at SF11 (Semtech, *SX1261/2*). The signal is literally quieter than the noise floor, and the receiver still reads it. This is the physical origin of LoRa's range, and its price is time: every symbol lasts 8.19 ms, which makes airtime the scarce resource the rest of the design must budget.

### 4.2 The cost of one packet

The number of symbols required to carry a payload follows Semtech's time-on-air formula, in which PL is the payload length in bytes, H is zero when an explicit header is present, DE is the low-data-rate optimization flag, CRC indicates the integrity checksum, and CR = 1 encodes the 4:5 coding rate. The numerator counts the bits to move, the denominator counts the bits each symbol carries, and the factor (CR + 4) prices the forward error correction (Semtech, *AN1200.13*):

$$N_{payload} = 8 + \max\left(\left\lceil \frac{8PL - 4SF + 28 + 16\,CRC - 20H}{4(SF - 2DE)} \right\rceil (CR + 4),\ 0\right)$$

Evaluating this for a representative 40-byte Meshtastic position packet at SF = 11, BW = 250 kHz, DE = 0, with the CRC enabled and a 16-symbol preamble, gives 48 payload symbols, and adding the preamble and the 4.25 synchronization symbols yields the total transmission time:

$$N_{payload} = 8 + \left\lceil \frac{8(40) - 44 + 28 + 16}{44} \right\rceil \times 5 = 48, \quad T_{pkt} = (16 + 4.25 + 48)\,T_{sym} = 68.25 \times 8.19\ \text{ms} \approx 0.56\ \text{s}$$

One position update costs roughly 0.56 seconds of air. This single number propagates through the entire system: it sets the fastest responsible reporting cadence (Section 6.3), it explains why the protocol cannot update once per second, and it is the unit in which the channel budget of Section 6.3 is denominated.

## 5. The Link Budget: From Datasheet Numbers to Street Range *(Cannon)*

A link budget adds every gain and subtracts every loss between transmitter and receiver, then asks whether what arrives still clears the receiver's sensitivity. The received power is the transmitted power plus both antenna gains minus the free-space path loss, which grows logarithmically with both distance and frequency:

$$P_{rx} = P_{tx} + G_{tx} + G_{rx} - FSPL, \qquad FSPL = 20\log_{10} d + 20\log_{10} f - 147.55 \;\; (d \text{ in m},\ f \text{ in Hz})$$

In this system the transmit power is +30 dBm, the stock quarter-wave antennas contribute approximately 0 dBi each, and the sensitivity at the chosen operating point is -131 dBm, giving a total budget of 161 dB of loss the link can absorb before decoding fails. Evaluating the path loss at the operating frequency of roughly 913 MHz for two reference distances shows how gently it grows:

$$FSPL_{1\,\mathrm{km}} = 60 + 179.2 - 147.55 = 91.7 \text{ dB} \Rightarrow P_{rx} = -61.7 \text{ dBm}; \qquad FSPL_{5\,\mathrm{km}} = 105.7 \text{ dB} \Rightarrow P_{rx} = -75.7 \text{ dBm}$$

Even at 5 km the free-space arrival of -75.7 dBm clears sensitivity by 55 dB. Streets, however, are not free space: terrain, buildings, foliage, and multipath fading consume margin unpredictably, so the design reserves 20 dB for the environment and quotes an honest design range of 3 to 5 km rather than the tens of kilometers the free-space arithmetic would permit. Margin converts to distance through the doubling rule implied by the 20 log term: every 6 dB of spare margin roughly doubles the achievable range. The measured desk-range baseline of roughly 6 dB signal-to-noise ratio sits 23.5 dB above the -17.5 dB demodulation floor, which is the reserve the system spends as the vehicles separate:

$$\frac{d_2}{d_1} = 10^{\Delta/20} \Rightarrow \Delta = 6 \text{ dB} \approx \times 2 \text{ distance}, \qquad \text{margin} = SNR_{meas} - SNR_{floor} = 6 - (-17.5) = 23.5 \text{ dB}$$

## 6. The Protocol Layer: Meshtastic and the Channel Contract *(Deo)*

### 6.1 The name is the frequency

The radios run Meshtastic, an open-source LoRa mesh networking firmware (Meshtastic Project, *Firmware*). Its most consequential design choice for this project is that a node stores no frequency at all. The United States band plan spans 902 to 928 MHz, which at 250 kHz per channel divides into 104 slots, and the firmware derives its slot by hashing the primary channel's name (Meshtastic Project, "Radio Settings"):

$$f_{slot} = f_{start} + \frac{BW}{2} + [\text{djb2}(name) \bmod N_{slots}] \cdot BW, \qquad N_{slots} = \frac{f_{stop} - f_{start}}{BW} = \frac{26 \text{ MHz}}{250 \text{ kHz}} = 104$$

The hash is djb2, which starts from the constant 5381 and, for each character, multiplies the running value by 33 and adds the character code. The formula can be checked against a public constant: djb2 of "LongFast," the default channel name, lands on slot 19, which is 906.875 MHz, the documented United States default. Our private channel name "Group9" hashes to slot 43, and substituting into the slot formula gives the frequency both radios must, and verifiably do, display:

$$f_{43} = 902.000 + 0.125 + 43 \times 0.250 = 912.875 \text{ MHz}$$

Two radios holding identical encryption keys but different channel names therefore sit megahertz apart and never exchange a bit. This team learned that lesson experimentally: the single longest outage of the semester was traced to a channel write that silently reverted on one radio, leaving both nodes healthy, both configurations apparently identical, and the pair mutually deaf on different slots. The durable diagnostic that emerged is that the frequency printed on each radio's OLED screen is the ground truth of the agreement, checked before every test.

### 6.2 Privacy on a public frequency

Slot 43 is public airspace; anyone may transmit or listen there. Privacy comes from the second term of the contract. All channel traffic is encrypted with AES-256 under a pre-shared key generated at channel creation and distributed between the radios only over USB, inside the provisioning URL; the key never crosses the air (Meshtastic Project, "Encryption"). Directed traffic such as on-demand position requests is additionally encrypted per node with Curve25519 public keys exchanged automatically in node-information broadcasts. An eavesdropper observes that packets exist, their timing, and their signal strength, because headers are cleartext, but recovers neither content nor location. The system's privacy is therefore layered exactly like its connectivity: the obscure frequency provides separation from the public mesh, and the key provides the actual secrecy.

### 6.3 Airtime as a shared budget

Because every packet costs 0.56 seconds of a channel only one node can occupy at a time, Meshtastic treats channel utilization as a managed resource and suppresses position broadcasts when utilization exceeds roughly 25 percent (Meshtastic Project, "Mesh Broadcast Algorithm"). The configured cadence must therefore be shown to fit. With both vehicles reporting every 10 seconds while moving, utilization is the node count times the packet time divided by the cadence:

$$U = \frac{N_{nodes} \cdot T_{pkt}}{T_{cadence}} = \frac{2 \times 0.56}{10} \approx 11\% \text{ (no relay)}; \quad \times 2 \text{ relayed} \approx 22\% \text{ vs the } 25\% \text{ throttle}$$

The second figure in the equation records a subtlety of running a mesh protocol with exactly two nodes. Meshtastic's default role re-broadcasts every overheard packet to extend multi-hop meshes, but with two nodes every relay is pure waste that doubles the spend to roughly 22 percent, brushing the throttle precisely when both vehicles move. Both radios therefore run the CLIENT_MUTE role, which transmits and receives normally but never relays; the change was measured to halve idle utilization from the 9 to 13 percent range to 5 to 6 percent. The full cadence contract is: positions every 10 seconds and 20 meters while moving, every 60 seconds while parked, device telemetry every 300 seconds, and node information on boot. Each periodic packet exists to prevent a named failure, and the telemetry heartbeat earns the next section.

### 6.4 Liveness is arithmetic

No wire connects the vehicles, so nothing can prove a silent partner is alive. Every liveness indicator in the system is a comparison between the current time and a stored last-heard timestamp. The stock firmware's definition of an online node is one inequality, with the constant defined in its node database source (Meshtastic Project, *Firmware*):

$$\text{online}(n) \;\Leftrightarrow\; t_{now} - t_{last\ heard}(n) < 7200\ \text{s}$$

The two-hour window is sized for an ecosystem of solar nodes that legitimately sleep, and it must exceed the longest honest silence or healthy nodes flap offline; its cost is that a dead radio can look alive for two hours. Because this system's cadence contract guarantees a packet at least every 300 seconds, the display layers far stricter thresholds on the same timestamps: a link-fresh tag within 90 seconds, a visibly degraded position estimate past 30 seconds, and a stale-data banner past 120 seconds. Several of the semester's most confusing symptoms, including online counters claiming a partner was last heard 72 days ago while text messages flowed normally, were ultimately stale stored timestamps being re-broadcast by a node without a GNSS fix, and every one of them yielded to the same analysis: the counter is not a connection, it is arithmetic on a timestamp, so find the timestamp that is lying.

### 6.5 The configuration as a written contract

Because agreement is the connection, the radio configuration is not a list of preferences but the contract itself, and this team treats it that way: every value is written down with its reason, applied identically to both radios, and verified after every change. Table 1 records the full operating configuration. The companion lesson, earned the hard way and examined in Section 9, is that applying a setting and having a setting are different things. The provisioning tool talks to the radio over a USB serial port, and closing that port toggles a control line that hard resets the processor; a write that arrives just before the reset can be lost before the radio saves it, while the tool happily reports success. The discipline that resulted is mechanical. After any configuration change, read every value back, confirm the OLED frequency screen shows 912.875 MHz, then power cycle the radio and confirm the value survived.

| Setting | Value | Reason |
|---|---|---|
| Region | US | Selects the 902 to 928 MHz band plan the slot arithmetic of Section 6.1 assumes |
| Primary channel | Group9, position precision 32 | Private slot 43; full-resolution positions, since the channel default truncates every fix to a grid of roughly 6 km |
| Modem preset | LONG_FAST (SF11, 250 kHz, 4:5) | The range-versus-rate compromise derived in Section 4 |
| Transmit power | +30 dBm | The legal ceiling; margin converts directly to range (Section 5) |
| Position broadcast, parked | 60 s | Presence at negligible airtime cost |
| Position broadcast, moving | 10 s and 20 m, both required | Tracking freshness inside the airtime budget of Section 6.3 |
| GNSS update interval | 10 s | Must not exceed the broadcast interval, or the radio re-sends stale fixes |
| Telemetry heartbeat | 300 s | Keeps the liveness arithmetic of Section 6.4 honest for about 0.17 percent airtime |
| Device role | CLIENT_MUTE | Two-node network; relaying is pure airtime waste (Section 6.3) |
| Bluetooth mode | NO_PIN | Lets the display pair without a keypad; bonded modes stop advertising and strand the display |
| Canned messages | 21 full-word presets | Conversation without typing; the firmware caps the whole set at 200 characters and this set uses 194 |

Table 1. Operating configuration applied identically to both radios.

## 7. The Display Node: Waveshare ESP32-S3-RLCD-4.2 *(Josiah)*

### 7.1 Hardware

The display node is a Waveshare ESP32-S3-RLCD-4.2 driver board: the same ESP32-S3 microcontroller class as the radio, driving a 4.2-inch, 400 by 300 pixel reflective LCD (Waveshare). A reflective panel modulates ambient light rather than emitting its own, which makes it brightest exactly where a dashboard instrument needs it, in direct sunlight, while drawing milliwatts and producing no glare at night. The board exposes the single button and USB-C port the firmware uses, and both units are enclosed in PETG cases printed from a community design, chosen over PLA because dashboard temperatures exceed PLA's glass transition.

### 7.2 What made it possible: the Client API over BLE *(Deo)*

The decision that made a custom display feasible alongside unmodified radio firmware is that Meshtastic publishes its phone-application interface as a documented Bluetooth Low Energy service, the Client API (Meshtastic Project, "Client API"). The radio exposes one GATT service with three characteristics: a ToRadio characteristic the client writes commands and outbound packets into, a

FromRadio characteristic the client reads repeatedly until empty, and a FromNum characteristic that notifies when new data is queued. Every payload is a protocol buffer, a compact binary serialization with published schemas. The display firmware therefore impersonates the phone app: it connects as a BLE central using the NimBLE-Arduino stack (h2zero), writes a configuration request, drains the radio's node database, and then drains every subsequent notification, decoding each FromRadio message with nanopb, a protocol-buffer decoder designed for microcontrollers (Aimonen). The radio cannot tell the difference between this team's 400-line protocol layer and the official application, which is precisely why no modification of the radio firmware, and none of the risk that would carry, was ever required.

Table 2 lists the complete Bluetooth interface. One service and three characteristics carry everything: configuration, positions, telemetry, node information, and text, all as protocol buffers.

| GATT item | UUID | Role |
|---|---|---|
| Service | `6ba1b218-15a8-461f-9fa8-5dcae273eafd` | The Meshtastic Client API service |
| ToRadio | `f75c76d2-129e-4dad-a1dd-7866124401e7` | Written by the client: commands and outbound packets |
| FromRadio | `2c55e69e-4993-11ed-b878-0242ac120002` | Read repeatedly until empty: the configuration dump and all inbound packets |
| FromNum | `ed9da18c-a800-4f66-a670-aa7547e34453` | Notifies the client that new data is queued |

Table 2. The Meshtastic Client API as exposed over Bluetooth Low Energy (Meshtastic Project, "Client API").

### 7.3 Inside the display firmware

The decoded stream feeds a deliberately simple architecture: a Bluetooth task writes and a rendering task reads a single mutex-protected state structure containing both vehicles' fixes, names, battery levels, and message history. The one-writer, one-reader discipline eliminates data races by construction, and because the shared structure is pure data with no hardware dependencies, the navigation mathematics compiles and runs on a desktop computer, where twelve unit tests exercise it against known coordinates. The firmware totals roughly 2200 hand-written lines of C and C++ plus the generated protocol-buffer decoding code, organized into a protocol layer (Bluetooth link, decoder, node table), a navigation layer (the mathematics of Section 8), and a rendering layer (three screen pages drawn at two frames per second).

Two of the firmware's smallest decisions carry the most design weight, and both are about honesty. First, the radio echoes a copy of every message its user sends back over the Bluetooth link, so a naive display would pop up a notification for the driver's own words; the decoder therefore discards any text whose origin is the local node, which guarantees that a banner on the screen is always news from the other vehicle. Second, a radio that has lost its GNSS fix keeps re-broadcasting its last stored position, and a naive display would treat each arrival as fresh; the node table therefore refuses to reset the data age when an identical position arrives again, so a vehicle sitting in a parking garage goes visibly stale

instead of invisibly wrong. Between genuine updates the display projects both vehicles forward by dead reckoning for smooth motion, but the same principle holds there: projection never resets the age, and past 30 seconds the direction dart is drawn hollow. The driver can always distinguish what the system knows from what it is guessing.

The display's single push button completes the interaction model. A short press cycles between the compass, status, and message pages; a long press clears the message history or resets the tracking state; and the board's second button transmits an on-demand position request to the partner, a directed packet that asks the remote radio to answer immediately rather than at its next scheduled broadcast. That request exercises the full protocol stack, including the per-node encryption of Section 6.2, which is how one of the subtler failures of Section 9 was discovered.

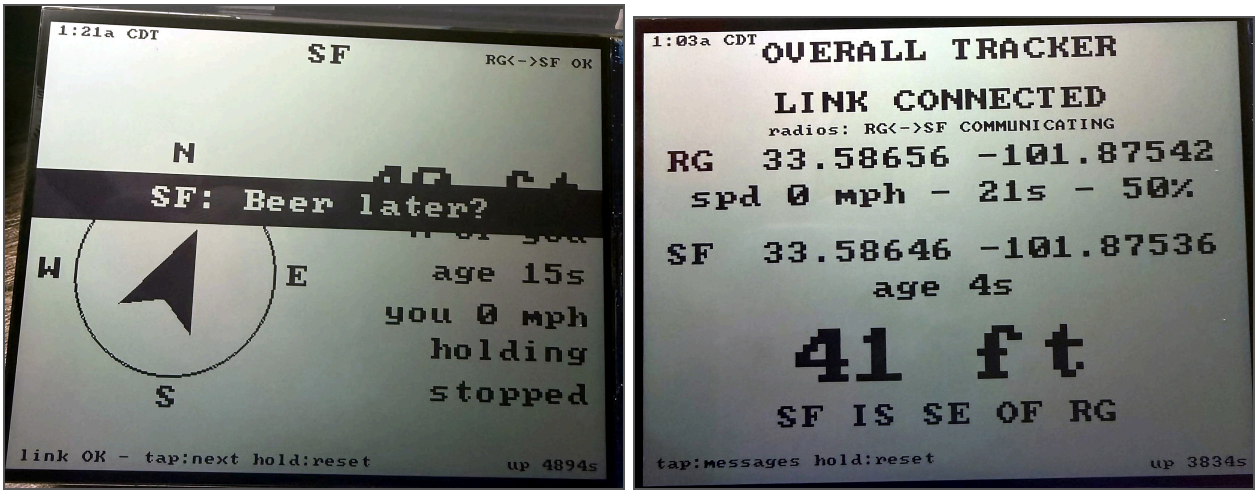



Figure 4. The display in operation: the compass page with an arriving message (left) and the status page listing both vehicles, their data ages, batteries, and separation (right).

## 8. Navigation Mathematics *(Josiah)*

Everything the driver sees is computed on the display from two stored fixes. Latitude φ and longitude λ are in radians, ψ is the vehicle's own course over ground, and R is the Earth's mean radius. The distance between the vehicles is the great-circle distance given by the haversine formula, whose intermediate quantity a measures the separation on the unit sphere:

$$a = \sin^2\frac{\Delta\varphi}{2} + \cos\varphi_1\cos\varphi_2\sin^2\frac{\Delta\lambda}{2}, \qquad d = 2R\arcsin\sqrt{a}, \qquad R = 6371\ \text{km}$$

During bench verification the two radios, sitting on one desk, reported fixes that this formula placed 12.5 meters apart, which is not an error in the formula but an honest picture of consumer GNSS scatter between two independent receivers; the figure tightened outdoors. The direction to the partner is the initial great-circle bearing, and subtracting the driver's own course and wrapping to plus or minus 180 degrees gives the angle the on-screen arrow makes with the vehicle's nose:

$$\theta = \operatorname{atan2}(\sin\Delta\lambda\cos\varphi_2,\ \cos\varphi_1\sin\varphi_2 - \sin\varphi_1\cos\varphi_2\cos\Delta\lambda), \qquad \alpha_{arrow} = \operatorname{wrap}_{\pm 180}(\theta - \psi_{course})$$

Radio updates arrive at most every ten seconds, but the screen renders twice per second. Between packets each fix is projected forward along its last course and speed, with the cosine of latitude correcting for the convergence of meridians, and with projection capped at 120 seconds:

$$\varphi(t) = \varphi_0 + \frac{v\,\Delta t \cos\psi}{R}, \qquad \lambda(t) = \lambda_0 + \frac{v\,\Delta t \sin\psi}{R\cos\varphi}, \qquad \Delta t \le 120\ \text{s}$$

The projection is cosmetic, and the design is careful to say so: the displayed data age is never reset by projection, and past 30 seconds the arrow is drawn hollow, so motion stays smooth while the symbology tells the truth about freshness. Finally, the display reports whether the gap is closing or opening through an exponential moving average of the distance derivative, sampled each 500-millisecond render tick; the 0.9 to 0.1 weighting settles in about five seconds, enough smoothing that GNSS jitter does not flap the label:

$$r_k = 0.9\,r_{k-1} + 0.1 \cdot \frac{d_{k-1} - d_k}{\Delta t}, \qquad \Delta t = 0.5\ \text{s}; \quad r > 0:\ \text{closing},\ \ r < 0:\ \text{apart}$$

## 9. Failure Modes and Lessons Learned

A design report that lists only successes hides most of its engineering. This section documents the five most instructive failures of the semester, each reproduced or root-caused on the bench before being written down. They share a family resemblance: in every case the hardware was healthy, the symptom pointed somewhere misleading, and the explanation was one of the agreements of Section 1 quietly broken. The rules this team now follows before every test were each purchased by one of these.

### 9.1 The silent frequency revert

The longest outage of the project presented as total mutual silence with zero channel utilization on both radios, while both configurations read back as identical. The root cause was the configuration write failure described in Section 6.5: a channel write had silently reverted on one radio, leaving the two nodes with the same key on different frequency slots, one on the private slot 43 and one back on the public slot 19, six megahertz away. The diagnostic lesson is that a read-back queries the intended configuration, while the OLED frequency screen reports the frequency the synthesizer is actually programmed to. The screen is the ground truth. The rule adopted is a two-second pre-flight check: both OLED screens must read 912.875 before any session.

### 9.2 Battery death erases configuration

Twice in one night, radios whose batteries had been allowed to run completely flat came back with reverted configuration, one losing only its channels and the other losing everything including its name and keys. The symptom is always the same healthy-looking silence as Section 9.1, which is why the same pre-flight check catches it. The operating rule is to never run the radios to empty, and the recovery is the full provisioning recipe of Section 6.5 followed by a power-cycle persistence check.

### 9.3 The half-alive radio

A radio recovering from a flat battery once ran for 18 hours with a lit screen and a valid GNSS fix while its transmitter and USB port were both frozen; nothing it displayed suggested a problem, and nothing it transmitted existed. The lesson generalizes beyond this project: a device's own user interface is not evidence of its health. Judge a radio by the other radio's last-heard age, never by its own screen. A full power cycle cleared the condition, and a clean boot announces itself on the channel within a minute.

### 9.4 The stale public key

After one radio was factory reset, broadcasts continued to flow in both directions, but directed packets, position requests and direct messages, timed out in exactly one direction with no error. The cause is the key model of Section 6.2: directed traffic is encrypted to the recipient's stored public key, the factory reset had regenerated that key, and the requester was still encrypting to the old one. The recipient dropped what it could not decrypt, silently, exactly as a secure system should. The fix is to clear the requester's node database and let the next node-information broadcast deliver the fresh key. The lesson is that asymmetric encryption fails asymmetrically, so a one-way failure of directed traffic with healthy broadcasts points at keys, not radio.

### 9.5 Counters that lie

The most stubborn class of symptom was liveness indicators contradicting reality: a partner showing as last heard 72 days ago in the middle of a working text conversation, and node lists counting long-dead entries as online after a reboot. Section 6.4 explains the mechanism, stale timestamps re-broadcast by fixless radios and stale node records reloaded from flash. The cure was threefold: purge stored positions so a fixless radio goes honestly quiet, synchronize clocks, and add the 300-second telemetry heartbeat so that fresh timestamps always exist. The design lesson echoes through Sections 6.4 and 7.3: every liveness indicator is arithmetic on a timestamp, so when an indicator lies, some timestamp is lying, and the job is to find which one.

## 10. Experimental Verification *(Victor and Cannon)*

Every major claim in this report has been exercised on the real hardware, and several of the system's rules exist only because an experiment in Section 9 failed first.

| Claim | Evidence |
|---|---|
| Two-way link on slot 43 | Round-trip traceroutes across multiple sessions, 5.75 to 7.25 dB SNR per direction at +30 dBm |
| Frequency-slot derivation | Both OLEDs display 912.875 MHz on channel Group9; the 2026-09-25 outage reproduced the mismatch case |
| Live tracking at vehicle cadence | A display tracked a real drive on first boot; the 10-second moving cadence held through the Week 4 drive test |
| On-demand position request | Both the command-line request and the display's button returned the partner's live fix within seconds |
| Messaging path | Preset texts delivered in both directions; the popup confirmed receiver-side only |
| Airtime management | Switching both nodes to CLIENT_MUTE halved measured idle utilization, from roughly 9 to 13 percent down to 5 to 6 percent |
| Navigation mathematics | Twelve host-side unit tests pass; bench distances match hand-computed haversine values |

The two-vehicle drive tests were run against acceptance metrics declared in advance, summarized in Table 3, with the measured outcomes recorded in Appendix C. Declaring the thresholds before the test, rather than after, is deliberate; it converts the drive from a demonstration into an experiment that can fail, which is the only kind that provides evidence. The range model check passed with margin: full tracking at 5.25 km against a 3 to 5 km design range.

| Metric | Acceptance threshold | Method |
|---|---|---|
| Partner position freshness | 95th percentile of displayed age no greater than 15 s while both vehicles move | Log displayed age each render tick during a 30-minute convoy drive |
| Reacquisition after separation | Fresh partner fix within 60 s of returning to range | Deliberately separate beyond range, return, and time the recovery |
| Silent loss | No interval over 120 s without an update while in convoy range | Post-drive inspection of both logs |
| Range model check | Measured SNR versus distance tracks the Section 5 prediction within the 20 dB environmental reserve | Stop at measured distances; record per-packet SNR both directions |

Table 3. Acceptance metrics for the two-vehicle drive test, declared in advance.

## 11. Future Work

Three extensions follow naturally from the architecture. The first is a third node: the protocol already supports many, and the only configuration change is reverting both radios from CLIENT_MUTE to the standard client role so packets relay again, at the airtime cost quantified in Section 6.3. The second is a fixed repeater on high ground for terrain that defeats the direct link, enabled by the same role change, with the Section 5 link budget applying unchanged to each hop. The third is powering radio and display from the ignition circuit, which would retire the battery-death failure class of Section 9.2 entirely. Beyond these, the range characterization of Table 3 would turn the Section 5 model from a design argument into a validated measurement.

## 12. Conclusion

The system works because each layer keeps one precise promise to the layer above it. Chirp spread spectrum promises that 11-bit symbols survive 17.5 dB below the noise floor, and charges 8.19 milliseconds per symbol for it. The link budget promises 161 dB of absorbable loss, and the design spends 20 dB of it on the real world to quote an honest 3 to 5 km. Meshtastic promises that a channel name, hashed, is a frequency, and that a key shared over a cable is privacy on public airspace. The Client API promises that anything that speaks its protocol buffers over three Bluetooth characteristics is a legitimate client, which is what admitted a custom sunlight-readable display without a line of radio firmware changing. And the display, holding the top of the stack, promises the driver honesty: every number on the glass is either a measurement with its age attached or an estimate drawn hollow. The failures of Section 9 were, without exception, one of these promises quietly broken and then restored, which is its own kind of confirmation that the layering is real. The result is two ordinary vehicles that know where each other are, anywhere on Earth with a view of the sky, with nothing between them but physics and agreement.

## References


1. Aimonen, Petteri. *Nanopb: Protocol Buffers with Small Code Size*. GitHub, github.com/nanopb/nanopb.
2. Espressif Systems. *ESP32-S3 Series Datasheet*. Version 2.0, Espressif Systems, 2024, www.espressif.com/en/products/socs/esp32-s3.
3. h2zero. *NimBLE-Arduino: A Fork of the NimBLE Stack for Arduino*. GitHub, github.com/h2zero/NimBLE-Arduino.
4. Heltec Automation. *WiFi LoRa 32 (V4) Product Documentation*. Heltec Automation, heltec.org/project/wifi-lora-32-v4.
5. Meshtastic Project. "Client API." *Meshtastic Documentation*, meshtastic.org/docs/development/device/client-api.
6. Meshtastic Project. "Encryption." *Meshtastic Documentation*, meshtastic.org/docs/overview/encryption.
7. Meshtastic Project. "Mesh Broadcast Algorithm." *Meshtastic Documentation*, meshtastic.org/docs/overview/mesh-algo.
8. Meshtastic Project. "Radio Settings." *Meshtastic Documentation*, meshtastic.org/docs/configuration/radio.
9. Meshtastic Project. *Meshtastic Device Firmware*. Version 2.7.26, GitHub, github.com/meshtastic/firmware.
10. Semtech Corporation. *AN1200.13: LoRa Modem Designer's Guide*. Semtech Corporation, 2013, www.semtech.com.
11. Semtech Corporation. *SX1261/2 Long Range, Low Power, Sub-GHz RF Transceiver Datasheet*. Revision 2.2, Semtech Corporation, 2023, www.semtech.com/products/wireless-rf/lora-connect/sx1262.
12. United States. *Code of Federal Regulations*, title 47, section 15.247. Electronic Code of Federal Regulations, www.ecfr.gov/current/title-47/section-15.247.
13. Waveshare Electronics. *ESP32-S3-RLCD-4.2 Development Board Wiki*. Waveshare, www.waveshare.com/wiki/ESP32-S3-RLCD-4.2.

## Appendix A: Acronyms and Abbreviations

Each term is expanded and then defined in the role it plays in this system.

| Term | Expansion and meaning in this system |
|---|---|
| ADC | Analog-to-digital converter; digitizes the down-converted signal ahead of the LoRa modem in Figure 2 |
| AES-256 | Advanced Encryption Standard with a 256-bit key; encrypts all channel traffic (Section 6.2) |
| BLE | Bluetooth Low Energy; the short-range in-vehicle link between each display and its radio (Section 7.2) |
| BW | Bandwidth; the RF channel width, 250 kHz in this design |
| C.F.R. | Code of Federal Regulations; title 47, section 15.247 governs this band |
| CR | Coding rate; the forward error correction ratio, 4:5 here |
| CRC | Cyclic redundancy check; the per-packet integrity checksum |
| CSS | Chirp spread spectrum; the LoRa modulation of Section 4 |
| dB, dBm, dBi | Decibel ratio; power referenced to one milliwatt; antenna gain referenced to an isotropic radiator |
| FS | Frequency synthesis; the transceiver mode in Figure 3 in which the PLL locks the carrier |
| FSK | Frequency-shift keying; the SX1262's alternative modem, unused in this design |
| GATT | Generic Attribute Profile; the service and characteristic structure of BLE (Table 2) |
| GNSS | Global navigation satellite system; the source of every position fix |
| LDO | Low-dropout regulator; one of the transceiver's two supply options in Figure 2 |
| LNA | Low-noise amplifier; the first stage of the receive chain in Figure 2 |
| LoRa | Long Range; Semtech's chirp spread spectrum modulation |
| NVS | Non-volatile storage; the flash region holding radio configuration, implicated in Section 9.2 |
| OLED | Organic light-emitting diode; the radio's small status screen, the ground truth of Section 9.1 |
| PA | Power amplifier; the final transmit stage in Figure 2 |
| PLL | Phase-locked loop; the frequency synthesizer the channel-name hash ultimately programs |
| RLCD | Reflective liquid crystal display; the sunlight-readable panel of Section 7.1 |
| SF | Spreading factor; bits per LoRa symbol, 11 here (also the callsign of radio 1) |
| SNR | Signal-to-noise ratio; signal strength relative to the noise floor, in dB |
| SPI | Serial Peripheral Interface; the four-wire bus between the ESP32-S3 and the transceiver |
| UUID | Universally unique identifier; names the BLE service and characteristics in Table 2 |

## Appendix B: Table of Symbols

The symbols below appear in the equations of Sections 4 through 8, listed in order of first use.

| Symbol | Meaning | Value or units |
|---|---|---|
| $f_{slot}$, $f_{start}$, $f_{stop}$ | Slot center frequency; band edges of the US plan | MHz; 902 and 928 MHz |
| $N_{slots}$ | Number of frequency slots in the band | 104 |
| $T_{sym}$ | Duration of one LoRa symbol | 8.19 ms at SF11, 250 kHz |
| $N_{payload}$ | Symbol count of the payload portion of a packet | 48 for the worked 40-byte packet |
| PL, H, DE, CRC, CR | Payload bytes; header flag; low-data-rate flag; checksum flag; coding-rate index | Dimensionless inputs to the time-on-air formula |
| $T_{pkt}$ | Total time on air of one packet | Approximately 0.56 s |
| $P_{tx}$, $P_{rx}$ | Transmitted and received power | dBm; +30 dBm transmit |
| $G_{tx}$, $G_{rx}$ | Antenna gains | Approximately 0 dBi each |
| FSPL | Free-space path loss | dB; 91.7 dB at 1 km, 913 MHz |
| S | Receiver sensitivity | -131 dBm at the operating point |
| U | Channel utilization | Fraction of time the channel carries packets |
| $t_{now}$, $t_{last\ heard}$ | Current time; timestamp of the last packet from a node | Seconds; the online window is 7200 s |
| $\varphi$, $\lambda$ | Latitude and longitude | Radians in the formulas of Section 8 |
| a, d, R | Haversine intermediate; great-circle distance; Earth mean radius | Dimensionless; meters; 6371 km |
| $\theta$, $\alpha$, $\psi$ | Bearing to partner; arrow angle relative to the nose; own course over ground | Degrees |
| v, $\Delta t$ | Speed and elapsed time in the dead-reckoning projection | m/s; seconds, capped at 120 s |
| $r_k$ | Smoothed closing rate at render tick k | m/s; positive means closing |

## Appendix C: Bench Measurement Record

The measurements below are the raw observations behind the evidence table of Section 10, recorded during bench and desk-range sessions across the semester. They are reproduced here so a reader can judge the claims against the data rather than against a summary of it.

| Measurement | Observed values | Notes |
|---|---|---|
| Round-trip traceroute SNR, desk range | 6.75 dB and 5.75 dB per direction on the verifying session; 5.75 to 7.25 dB across later sessions | +30 dBm, antennas meters apart; confirms the link and gives the margin baseline of Section 5 |
| Channel utilization, default client role | 9 to 13 percent at idle | Two nodes relaying each other's broadcasts |
| Channel utilization, CLIENT_MUTE | 5 to 6 percent at idle; below 4 percent typical in later sessions | Relaying disabled; roughly half the default role's spend |
| GNSS scatter between adjacent radios | 13 to 45 ft displayed between two radios on one desk; 41 ft on the session of Figure 4 | Two independent consumer receivers; tightens outdoors and in motion |
| Position-request round trip | Partner's live fix returned within seconds of the request, at full precision | Exercised from the command line and from the display's button |
| Fix acquisition, open sky | Five-satellite fixes typically inside one minute from power on | Both radios, repeated across sessions |
| Heartbeat effect on liveness | With the 300 s telemetry heartbeat, no false offline event observed in subsequent bench sessions | Before the heartbeat, counters lied within hours on fixless radios (Section 9.5) |
| Display link freshness | Display reconnects to its radio during the radio's first advertising minute; link tag holds through a full session | Ignition-on power sequencing, both units |
| Open-road range, full health | LINK CONNECTED at 3.26 mi (5.25 km), both coordinate sets live on screen; the displayed distance matches the latitude delta exactly (0.04719 deg = 3.26 mi) | 2026-10-05, N County Road 2000; in-cabin antennas, partner parked; Find My ground truth 4.1 road miles |
| Open-road range, stale edge | DATA STALE at 3.75 mi (6.03 km), age 2 min, and a message send failed; flagged honestly on both devices | The reliable edge for this configuration, not a physics limit |
| In-town stale onset | Near 0.5 mi with the same hardware | 2026-10-04 night drive; a factor of seven from environment alone |
| Reacquisition inbound | Ages back in the teens by 0.85 mi; dart at 12 o'clock at 37 mph, no operator action | Return leg of the range test |

Table 4. Bench and road measurement record. Desk-range SNR values are not range claims; they anchor the margin arithmetic of Section 5.

Two figures tie the record together. The 6 dB desk-range SNR sits 23.5 dB above the -17.5 dB demodulation floor, roughly four distance doublings of reserve by the rule of Section 5, which was the basis for believing the 3 to 5 km design range in advance. The road then agreed: the 2026-10-05 range test measured full health at 5.25 km and the reliable edge by 6.03 km with antennas still inside the vehicles, meeting the top of the design range and exceeding it by about a kilometer. This satisfies the range model check of Table 3, and the in-town versus open-road comparison, 0.5 mi against 3.75 mi on identical hardware, is the 20 dB environmental reserve made directly visible.

## Appendix D: The Canned Message Set

Messages are sent from the radio's button interface, so the design problem is conversation without typing. The stock firmware stores the entire set as one string capped at 200 characters, a constraint that forces economy: this set spends 194 of them on 21 full-word messages. The design rule was that every question in the set must have its answer in the set, so the list functions as a closed conversation kit rather than a pile of phrases.

| Group | Messages | Design intent |
|---|---|---|
| Quick replies | OK · Yes · No · Hello | One-press answers to anything |
| Checking in | How are you? · I am good | A question and its answer, paired |
| Safety | Help! · Are you hurt? | The two messages that justify the rest of the system |
| Position | Send position | Backs up the display's request button over the text channel |
| Driving | On my way · Wait there · I am here · Follow me · Slow down · Stop · Turn left · Turn right · Turn back | Convoy coordination in full words, no abbreviations to misread at a glance |
| Social and status | Beer later? · Low battery · Going home | Trip-end coordination; the battery warning feeds the rule of Section 9.2 |

Table 5. The 21-message preset set carried identically by both radios.

The set was applied to both radios over USB and verified by read-back, and the delivery path was confirmed end to end: a message selected on one radio appears as a banner on the partner's display within seconds, is stored in the display's history page, and never notifies the sender's own screen, per the honesty rule of Section 7.3.